PE-JV-003

Aula 08 - Engenharia de Software com IA & Agentes

Aquecimento

Kahoot

Revisão da Aula 07: BOLA contra UUID, 404 contra 403 e Oráculos de Enumeração, JWT com JWKS e Algoritmo Fixo, Mitigação de Validação Offline e Zero Trust.

Aguarde as orientações do professor.

Engenharia com IA: O Dilema da Produtividade

O time adotou agentes e assistentes de código para acelerar o desenvolvimento de novos microsserviços.

O choque de realidade:
Escrever código nunca foi o principal gargalo da engenharia. Em poucas semanas, a equipe enfrentou:

1. **Código sem Especificação:** Prompts livres geram classes que compilam, mas violam invariantes de domínio, quebram convenções e acumulam débito técnico.
2. **Fadiga de Revisão:** Pull Requests extensos com código gerado por IA sobrecarregam os seniores, que gastam mais tempo caçando alucinações sutis do que construindo arquitetura.
3. **Agentes sem Ferramentas e Contexto:** O assistente opera no escuro. Ele não roda testes, não inspeciona schemas do banco e inventa chamadas a métodos inexistentes.

O problema:
Como passar da geração superficial de código para uma esteira de alta produtividade orientada por especificações formais (SDD), skills e ferramentas?

A IA na Aplicação: O Desafio em Produção

Além de usar IA no desenvolvimento, o marketplace colocou agentes no ar para explicar recusas de transações e triar chamados no aplicativo.

O incidente na operação:
Sob 5.000 TPS, o comportamento não determinístico da IA gerou falhas graves:

1. **Vazamento de PII:** Clientes digitaram dados pessoais no texto livre de suporte. O modelo misturou nomes e CPFs em respostas e logs.
2. **Ação Indevida por Injection:** Instruções maliciosas enviadas no campo de observação acionaram ferramentas de estorno sem validação de regras no backend.
3. **Degradação de Throughput:** Chamadas síncronas ao provedor de IA geraram timeouts de 15 segundos e esgotaram pools de conexão.

A Missão do Dia

Nesta aula com foco em arquitetura, produtividade e debate técnico:

- Dominar o **Spec-Driven Development (SDD)**: transformar regras de negócio em especificações rígidas e contratos determinísticos para orientar agentes.
- Desenhar a **Arquitetura de Agentes e Skills Modulares**: delimitar onde o código backend calcula e persiste, e onde o modelo apenas redige ou classifica.
- Estruturar integrações via **Model Context Protocol (MCP)**: conectar agentes a ferramentas reais (banco, testes, APIs) com isolamento estrito de privilégios.
- Projetar salvaguardas de **Contexto e Privacidade**: eliminar vazamento de PII em texto livre e mitigar riscos críticos do OWASP Top 10 for LLM.
- Estabelecer a estratégia de **Testes e Evals**: separar testes determinísticos de montagem de prompt com dublês da avaliação contínua de acurácia (*evals*).

Sala de Aula Invertida: A Pesquisa

Acesse os links oficiais compartilhados no chat e investigue:

1. **Spec-Driven Development (SDD):** Por que prompts soltos falham em projetos reais? Como especificações formais e schemas JSON reduzem alucinações e direcionam agentes?
2. **Código vs Modelo:** Quais decisões de um sistema precisam ficar sob controle determinístico do backend Java e quais podem ser delegadas ao LLM?
3. **Model Context Protocol (MCP):** O que é o protocolo MCP, qual problema ele resolve na exposição de ferramentas e por que não se confunde com rodar modelos locais?
4. **Privacidade e OWASP LLM:** Como sanitizar dados pessoais (PII) em texto livre antes de compor o prompt? Como barrar ataques de Prompt Injection direta e indireta?
5. **Estratégia de Avaliação (Evals):** Como testar clientes de IA com dublês determinísticos e como medir a precisão de recuperação no RAG?

O Desafio Arquitetural: Engenharia de Harness

Fundamente suas decisões técnicas para o debate com a turma:

1. **"Construir a Coisa que Constrói a Coisa":**
    - Por que o papel do engenheiro migrou de digitar código manual para desenhar e calibrar o **harness** onde os agentes de IA operam de forma autônoma?
    - O que acontece na esteira de desenvolvimento quando o time aposta em prompts soltos em vez de especificações formais (SDD) e guardrails rígidos no repositório?
2. **A Anatomia do Harness e Feedback Loops:**
    - Quais peças formam um harness confiável no ecossistema backend (contratos de API, MCP, regras de contexto, linters e checadores de schema)?
    - Como desenhar loops de feedback determinísticos e rápidos para que o agente rode testes locais, detecte quebras e se autocorrija antes de abrir um PR?
3. **Manutenção do Harness, Governança e Limites:**
    - Ao manter o harness no dia a dia, a quais riscos o time deve se atentar (poluição de contexto, privilégios excessivos a ferramentas e degradação de regras)?
    - Em operações transacionais críticas, como o harness garante que o código determinístico mantenha o controle e impeça decisões arbitrárias do modelo?

Debate Técnico

É hora de debater com o comitê de engenharia (Staff Engineer):

- **O Papel do Engenheiro Sênior:** Se os agentes já escrevem a maior parte do código da aplicação, qual passa a ser o diferencial técnico do desenvolvedor? O que significa construir a coisa que constrói a coisa?
- **Harness Frouxo vs Alucinação:** Quando o agente gera código que viola padrões ou inventa bibliotecas, onde está a falha raiz: na capacidade do modelo ou na ausência de restrições no harness?
- **Feedback Loops Determinísticos:** Por que o harness precisa fornecer ao agente ferramentas de verificação rápida (linters, checadores de schema, testes locais) antes de qualquer revisão humana?
- **Governança de Contexto:** O que acontece com a assertividade do agente se o repositório estiver entupido de regras genéricas e documentações defasadas? Como manter o sinal alto e o ruído baixo?
- **Menor Privilégio em Ferramentas (MCP):** Ao equipar agentes com acesso a bancos de dados e APIs, quais barreiras do harness impedem comandos destrutivos ou vazamento de segredos?
- **A Fronteira Transacional:** Por que o modelo pode redigir a justificativa de uma recusa, mas jamais calcular valores ou decidir a aprovação de uma transação no backend?

Mesa Redonda Técnica & Simulação Sênior

Estrutura da Atividade:

- **Mesa Redonda de Engenharia:** Condução coletiva das decisões arquiteturais da Aula 08 (Engenharia de Harness, SDD, MCP e Governança) atuando como comitê técnico.
- **Simulação de Entrevistas Sênior:** Treino aberto em plenária com perguntas conceituais de alto nível sobre arquitetura de software, produtividade com IA e mitigação de riscos.

Dinâmica da Aula:

Assim como na Aula 07, não teremos escrita de documento de System Design ou entrega de código hoje. O tempo é 100% dedicado à discussão de trade-offs e à preparação para a banca final.

Próxima Aula: Defesa Técnica & Autoavaliação

Chegamos ao encontro final da nossa jornada de engenharia backend. Toda a arquitetura desenvolvida ao longo dos encontros será defendida diante da banca técnica.

A Dinâmica da Aula 09:

- **Banca de Entrevistas Técnicas:** Simulação de entrevista sênior sorteando questões dos eixos abordados (Java 21, Resiliência, Dados, Microsserviços, CI/CD, Observabilidade, Segurança e IA).
- **Consolidação do Projeto:** Fechamento dos diários de bordo e projetos individuais construídos na jornada.
- **Autoavaliação e Rubricas:** Avaliação de maturidade técnica e feedback formativo para ingresso em times de engenharia de alta escala.

Referências da Aula 08 (Engenharia com IA)

Links oficiais para estudo e aprofundamento:

- 📄 **Engenharia de Harness:** Construir a coisa que constrói a coisa
- 📄 **Building Effective Agents:** Anthropic Research Guide
- 📄 **Model Context Protocol (MCP):** Especificação Oficial MCP
- 📄 **Spec-Driven Development:** Thoughtworks Technology Radar
- 📄 **Spring AI Reference:** Spring AI Documentation
- 📄 **OWASP Top 10 for LLM:** OWASP GenAI Security Project

Obrig.ADA